

บทที่ 3

Servlet, JSP และ Java Bean

3.1 เซิร์ฟเล็ต (Servlet)

Servlet คือเทคโนโลยี ที่เกิดขึ้นมาจาก Java เพื่อใช้งานทางด้านการสร้าง โปรแกรมในรูปแบบ CGI สำหรับทำงานบนเว็บเซิร์ฟเวอร์ เพื่อให้ผลการแสดงเว็บสู่ผู้อ่าน มีการเปลี่ยนแปลงตามการใช้งาน (On The Fly) ซึ่งเป็นเทคโนโลยีที่สืบเนื่องมาจากเหตุผลดังนี้

- ความต้องการให้หน้าเว็บเปลี่ยนตามข้อมูลที่ผู้อ่านส่งให้ เช่น เว็บที่ให้บริการระบบค้นหา (Search Engine)ทั้งหลาย ที่ได้รับข้อมูลจากผู้ใช้ คือข้อความที่ต้องการค้นหา และแสดงผลลัพธ์หน้าเว็บ ตามข้อมูลที่ค้นหาเจอแก่ผู้ใช้
- ความต้องการให้หน้าเว็บเปลี่ยนตามข้อมูลที่มีการเปลี่ยนแปลงบ่อยครั้ง เช่น เว็บที่เกี่ยวข้องกับการรายงานข่าว หรือสภาพอากาศ จะเห็นว่าข้อมูลที่รายงานสู่ผู้ใช้ ดังกล่าว มีการเปลี่ยนแปลงอยู่ตลอดเวลา อาจเป็นรายชั่วโมง หรือต่อนาทีก็ได้
- ความต้องการให้หน้าเว็บเปลี่ยนตามระบบฐานข้อมูล เช่น ระบบควบคุมคลังสินค้า ที่อาศัยการเก็บข้อมูลในระบบฐานข้อมูล สามารถแสดงผลจำนวนสินค้า ตามข้อมูลที่มีอยู่จริงในฐานข้อมูล ออกสู่ผู้ใช้นระบบเว็บ

3.1.1 ข้อแตกต่างของ Servlet เมื่อเทียบกับ CGI Servlet เป็นเทคโนโลยี ที่มีรูปแบบการทำงาน เหมือนกับ เทคโนโลยี CGI ที่มีการใช้งานอยู่ แต่เนื่องจาก Servlet ถูกพัฒนามาจากเทคโนโลยีจาวาอีกที ดังนั้นต่อไปนี้เป็นข้อเปรียบเทียบซึ่งแสดงให้เห็น ถึงความแตกต่างระหว่าง Servlet และ CGI

3.1.1.1 ประสิทธิภาพ การทำงานของ CGI จะมีการสร้างโปรเซส (Process) ใหม่ทุกครั้ง ที่มีการเรียกใช้งานจากผู้ใช้นั้น หากมีการเรียกใช้งาน จำนวนหลายครั้งในช่วงเวลาเดียวกัน ก็จะมีจำนวนโปรเซสเกิดขึ้นเท่ากับจำนวนครั้งที่เรียกใช้งาน โปรเซสจะมีขนาดและรูปแบบการทำงาน ที่เหมือนกัน ทำให้กินเวลาในการทำงานตามจำนวนโปรเซส แต่สำหรับ Servlet แล้ว ระบบเวอร์ชวลแมชีน (Java Virtual Machine) จะถูกดึงขึ้นมาทำงานก่อน เพื่อทำให้เกิดโปรเซสตามลักษณะการเรียกใช้ของผู้ใช้เช่นกัน แต่การทำงานของ Servlet จะเป็นรูปแบบเธรด(Thread) แทนที่จะเรียกว่าโปรเซส ซึ่งแต่ละเธรดขนาดการทำงานเล็กกว่าเดิม และกินระยะเวลาการทำงานน้อยกว่า จึงทำให้รูปแบบของ Servlet มีประสิทธิภาพกว่านั่นเอง

3.1.1.2 ความสะดวก หากคุณสามารถเขียนโปรแกรม ด้วยภาษา C, C++ หรือ Java อยู่แล้ว จะเกิดความสะดวกและรวดเร็วในการสร้าง Servlet มากกว่า แทนที่จะเริ่มศึกษาภาษาคำสั่งใหม่ๆ และ Servlet ก็มีความสามารถเพียงพอสำหรับการใช้งานบนเว็บอยู่แล้ว เช่น การรับข้อมูลจากฟอร์ม, การอ่านและตั้งค่า HTTP Header, การทำงานกับ Cookie, การทำงานแบบ Session และอื่นๆอีกมาก

3.1.1.3 ความสามารถ Servlet มีข้อดีที่เห็นได้ชัดคือ สามารถติดต่อกับเว็บเซิร์ฟเวอร์ได้โดยตรง ในขณะที่ CGI ปกติทำไม่ได้ (ยกเว้นรุ่นใหม่ๆ) และ Servlet ยังสามารถแชร์ข้อมูลการทำงานร่วมกับโปรแกรมอื่นๆได้ด้วย

3.1.1.4 การโยกย้าย เนื่องจาก Servlet ถูกสร้างมาจาก Java ซึ่งเป็นเทคโนโลยี ที่สามารถถูกใช้งาน ณ. ที่ใดก็ได้ โดยไม่จำกัดรูปแบบ ดังนั้น หากพัฒนา Servlet เพื่อใช้งานแล้ว สามารถที่จะโยกย้ายผลผลิตไปยัง ระบบอื่นๆได้ โดยไม่ต้องปรับปรุงหรือเปลี่ยนแปลง

3.1.1.5 ไม่แพง ในปัจจุบันเว็บเซิร์ฟเวอร์ที่สนับสนุนการทำงาน Servlet มีอยู่มากมาย ทั้งที่เป็นแบบฟรี(ดาวน์โหลด) และซื้อ-ขายกัน นอกจากนี้ ในอนาคตแผนการผลิตเว็บเซิร์ฟเวอร์อื่นๆ ยังมีการสนับสนุนการใช้งาน Servlet อีกด้วย

3.1.2 โครงสร้างของ Servlet การรับส่งข้อมูลโดยใช้เว็บ ผ่านทางแท็กฟอร์ม <FORM ...> มีรูปแบบการส่งอยู่ 2 ลักษณะคือ GET และ POST โดยที่การส่งแบบ GET เป็นรูปแบบการส่งข้อมูล ผ่านทาง URL (ช่องพิมพ์ชื่อเว็บหรือเว็บแอดเดรส) ซึ่งผู้ใช้อาจจะพิมพ์ได้เองโดยตรง เช่น <http://www.javacentrix.com?user=1245&pwd=4444> หรือฟอร์มที่แท็ก กำหนดแอตทริบิวต์ METHOD เป็น GET เช่น <FORM METHOD="GET" ... เป็นต้น (ตัวอย่างที่เป็นตัวหนาคือการส่งข้อมูลผ่านฟอร์มในแบบ GET)

สำหรับการส่งข้อมูลผ่านฟอร์มในแบบ POST คือการกำหนดแท็กฟอร์มให้มีแอตทริบิวต์ METHOD เป็น POST เช่น <FORM METHOD="POST" ... เป็นต้น

3.1.3 การใช้ Servlet ส่งข้อมูลรูปแบบ HTML แน่นอนว่าการสร้าง Servlet ขึ้นมาใช้งาน ไม่ได้มุ่งให้ส่งตัวอักษรเหมือนกับตัวอย่างที่ยกมา แต่ความต้องการคือ ให้สามารถส่งข้อมูลเว็บ ซึ่งก็คือข้อมูลในรูปแบบ HTML นั่นเอง ดังนั้นการส่งข้อมูล HTML ที่สำคัญ จำเป็นต้องส่งข้อมูลบอกลักษณะของรูปแบบ ให้กับผู้ใช้ (ซึ่งเป็นข้อมูลที่บราวเซอร์ได้รับ) ว่าเป็นข้อมูลรูปแบบ HTML ซึ่งเรียกกันว่า Content-Type แบบ text/html การส่งข้อมูลบอกบราวเซอร์นี้ ทำได้โดยการเรียกใช้เมธอด setContentType ของคลาส HttpServletResponse แล้วส่งข้อความ text/html เป็นอาร์กิวเมนต์ไป

3.1.4 การอ่านข้อมูลจากเว็บ สำหรับการใช้งานเว็บผ่านบราวเซอร์ โดยทั่วไปแล้ว ปกติในช่องที่ใช้กำหนดข้อมูล URL จะประกอบด้วยชื่อเซิร์ฟเวอร์ ชื่อไดเรกทอรี และชื่อไฟล์เว็บเพจที่ต้องการดู เช่น หากข้อมูล URL คือ <http://www.javacentrix.com/tutorials/api/index.htm> นั้นหมายถึงต้องการอ่านเว็บเพจที่ชื่อ index.htm จากไดเรกทอรี tutorials/api จากเว็บเซิร์ฟเวอร์ที่ชื่อ www.javacentrix.com

สำหรับ URL นั้นนอกจาก จะใช้ชี้ตำแหน่งไฟล์ที่ต้องการเรียกดูแล้ว ยังสามารถใช้สำหรับส่งข้อมูลให้กับระบบเซิร์ฟเวอร์ เพื่อใช้ในการทำงาน หรือดำเนินการอย่างใดอย่างหนึ่ง ลักษณะ URL ประเภทนี้ หลังจากชื่อไฟล์แล้ว จะตามด้วยเครื่องหมายคำถาม ? และข้อมูลที่ต้องการส่งไปให้เซิร์ฟเวอร์ ซึ่งเรียกโดยทั่วไปว่าการส่งข้อมูลแบบ GET โดยมีรูปแบบคือ URL?Parameter1=Data1&Parameter2=Data2

จากรูปแบบการส่งข้อมูลผ่าน URL โดย Parameter1 และ Parameter2 คือชื่อพารามิเตอร์ที่ถูกเซิร์ฟเวอร์อ้างอิงได้ ในขณะที่ Data1 และ Data2 คือข้อมูลที่อยู่ในพารามิเตอร์แต่ละตัว หากต้องการส่งข้อมูลมากกว่า 2 ตัว ก็จะเพิ่มต่อท้ายไปได้เรื่อยๆ เช่น

URL?Parameter1=Data1&Parameter2=Data2&Parameter3=Data3&Parameter4=Data4

3.2 เจเอสพี (Java Server Page : JSP)

Java Server Page หรือที่เรียกกันง่าย ๆ ว่า JSP เป็นเทคโนโลยีที่เป็นผลรวม ของการนำเอาหลักการ สร้างเว็บแบบสแตติกด้วย HTML มารวมกับการสร้างเว็บแบบไดนามิก ก่อให้เกิดการแสดงผลเว็บ ที่มีการ เปลี่ยนแปลงข้อมูลได้ตามการใช้งานของผู้ใช้ เห็นได้ว่าหากพูดถึงหลักการนำเสนอเว็บแล้ว เป็นหลักการที่ทำให้เว็บมีการเปลี่ยนแปลงข้อมูล เช่นเดียวกับ ที่ใช้โปรแกรมจำพวก CGI ทั่วไป แต่ในความหมายของ JSP แล้ว เป็นขั้นตอนและวิธีการสร้างที่ทำร่วมกับคำสั่ง HTML และเก็บบันทึกไว้ในไฟล์เดียวกัน แต่ในขณะที่ CGI เป็นโปรแกรมที่ผลิต HTML ให้เมื่อถูกสั่งให้ทำงาน

```
<HTML><HEAD><TITLE>Show JSP Page</TITLE></HEAD>
<BODY>
<% out.println("Hello Java Server Page"); %>
</BODY>
</HTML>
```

จากตัวอย่างด้านบนเป็นเพียงส่วนหนึ่งของ JSP ที่ถูกใส่ไว้ร่วมกับคำสั่ง HTML สังเกตจากตัวหนาที่ อยู่ใน แท็กพิเศษ <% ... %> ซึ่งเป็นคำสั่งในรูปแบบ JSP ทำให้ส่วนดังกล่าวสร้างข้อมูลในแบบไดนามิก ไว้ใน เว็บเพจตามคำสั่ง ไฟล์ดังกล่าวจะบันทึกไว้ในนามสกุล .jsp จาก HTML เดิม

รูปแบบดังกล่าวไม่สามารถทำได้สำเร็จตามเทคโนโลยี JSP หากไฟล์ตัวอย่างด้านบน ไม่ได้ถูกเก็บไว้ในเว็บเซิร์ฟเวอร์ที่สนับสนุนการทำงานของ JSP ข้อความที่เป็นไดนามิกนั้น ถูกสำรวจเพื่อประมวลผล และใส่ลงในตำแหน่งดังกล่าว ด้วยความสามารถของ เซิร์ฟเวอร์ที่สนับสนุน JSP ก่อนส่งกลับไปให้ผู้ใช้งาน โดยผู้ใช้งานจะได้รับข้อความ Hello Java Server Page เท่านั้นโดยไม่เห็นคำสั่ง JSP เลย

3.2.1 ข้อแตกต่างของ JSP เมื่อเทียบกับเทคโนโลยีอื่นๆ JSP ไม่ได้เป็นเทคโนโลยีเดียว ในปัจจุบันที่สามารถทำให้เว็บ แสดงข้อมูลในแบบไดนามิกได้ มีเทคโนโลยีอื่นๆที่ทำงานในลักษณะนี้หลายแบบ เนื้อหาส่วนนี้ เป็นการแนะนำให้ผู้อ่านได้เห็นข้อแตกต่างบางประการ เมื่อเทียบกับ JSP

- Active Server Page(ASP) ASP เป็นเทคโนโลยีที่เหมือนกับ JSP แต่เป็นเทคโนโลยีที่พัฒนามาจาก บริษัทไมโครซอฟต์ ผู้ผลิตระบบปฏิบัติการวินโดวส์ที่รู้จักกัน โดยทั่วไป ลักษณะ JSP มีรูปแบบที่แตกต่างกันเด่นชัด 2 ประการคือ ประการแรก JSP สามารถสร้างได้จาก ภาษาจาวา ต่างจากภาษา VB หรือภาษาใดๆ ซึ่งเป็นข้อกำหนดเฉพาะของ ไมโครซอฟต์ ประการที่สอง JSP ซึ่งเป็นผลพวงมาจากประการแรกคือ สามารถโยกย้ายการทำงาน ไปใช้งานระบบปฏิบัติการใดๆก็ได้ ที่มีการใช้งานกันอยู่ในปัจจุบัน โดยไม่ต้องแก้ไขหรือเปลี่ยนแปลงใดๆ ในขณะที่ ASP ก็โยกย้ายได้เช่นเดียวกัน แต่ต้องอยู่ในระบบปฏิบัติการที่ ไมโครซอฟต์กำหนดขึ้นเท่านั้น

- Servlet JSP สามารถทำงานได้เช่นเดียวกับที่ Servlet ทำได้ แต่มีจุดเด่นมากกว่าที่ JSP มีความสะดวกในการสร้าง และเปลี่ยนแปลง มากกว่า เพราะทำโดยตรงที่ไฟล์ HTML มากกว่าการที่ต้องลงมือเขียนคำสั่งภาษาจาวา โดยตรงเหมือนกันสร้าง Servlet ซึ่งต้องมีการนำไปคอมไพล์ก่อนการนำไปใช้งาน
- Server-Side Includes(SSI) SSI คือรูปแบบการสร้างไคนามิกเว็บเช่นเดียวกัน แต่รูปแบบการทำงานคือ การนำเอาข้อมูลที่มีอยู่แล้ว บนเซิร์ฟเวอร์นำมาประกอบใส่ ในเว็บเท่านั้น ต่างจาก JSP ซึ่งสามารถนำข้อมูลมาใส่ แต่มีรูปแบบในการทำงาน มีรูปแบบในการประมวลผล หรือการเรียกโปรแกรมภายนอก เช่น Servlet มาช่วยในการทำให้ข้อมูล มีรูปแบบต่างๆ เช่น ดึงข้อมูลมาจากระบบฐานข้อมูล (Database) นอกจากข้อมูลที่มีอยู่บนเซิร์ฟเวอร์เช่นที่ใช้ใน SSI เท่านั้น
- JavaScript เป็นการทำเนื้อหาไคนามิกให้กับเว็บ แต่รูปแบบการทำงานเกิดขึ้นของ JavaScript เกิดจากการประมวลผลและดึงข้อมูล ที่มีอยู่บนเครื่องผู้ใช้ หรือไคลเอนต์เท่านั้น มารวบรวมเนื้อหาในเว็บ ต่างกับ JSP ที่เป็นการทำไคนามิก แต่ข้อมูลถูกสร้าง และดึงมาจากระบบเซิร์ฟเวอร์ ที่มีหลากหลายมากกว่า และทำให้เนื้อข้อมูลเป็นเนื้อเดียวกัน เมื่อผู้ใช้เรียกดู แต่ JavaScript เนื้อข้อมูลเป็นของเครื่องผู้ใช้เอง ซึ่งผู้ใช้ต่างคน(ต่างเครื่อง) ก็จะให้ข้อมูลที่คล้ายกัน คือไม่เหมือนกันทั้งหมด
- Static HTML หรือสแตติก HTML เน้นอนข้อนี้ คงเห็นได้ชัดเจนที่ว่า สแตติก HTML ให้เนื้อข้อมูลที่คงที่ตลอดเวลา ไม่ว่าจะถูกเรียกดูในเวลาใดๆ (ยกเว้นมีคนมาแก้ไขคำสั่ง HTML) ในขณะที่ JSP ทำให้เนื้อข้อมูลมีการเปลี่ยนแปลงตามการใช้งาน หรือเครื่องผู้ใช้ โดยไม่ต้องเปลี่ยนแปลงแก้ไขคำสั่ง JSP

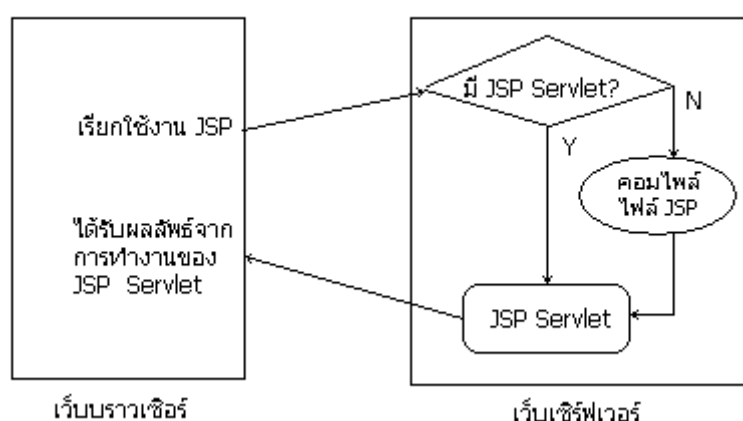
	CGI/Perl	Mod_Perl	ASP	JSP
เว็บเซิร์ฟเวอร์	Any Web server	Apache Web Server	Microsoft IIS or Personal Web Server	Any Web server, including Apache, Netscape, IIS today
โยกย้ายการทำงาน	No	No	No	Yes
ลักษณะนำกลับมาใช้	No	No	No	Yes
สคริปต์หรือภาษา	C, Perl	Perl	VBScript, JScript	Java
ปกป้องการใช้งานหน่วยความจำ	Yes	No	No	Yes
สนับสนุนการทำงานร่วมในหลายโปรเซส	No	Yes	Yes	Yes

ตารางที่ 3-1 เปรียบเทียบเทคโนโลยี JSP กับเทคโนโลยีอื่น

3.2.2 หลักการทำงานของ JSP และ ไคเร็กทีฟ JSP (Java Server Page) เป็นผลพวงมาจาก เทคโนโลยีของ Java Servlet ซึ่งเป็นหลักการสร้างระบบ การประมวลผลเว็บ ที่ทำงาน โดยอาศัยทรัพยากร (เมโมรี, ซีพียู, ดิสก์) ของเว็บเซิร์ฟเวอร์ ก่อนจะส่งผลลัพธ์สู่ผู้ใช้ ผ่านระบบเน็ตเวิร์กหรืออินเทอร์เน็ต โดยผู้สร้าง JSP สามารถสร้างโดยการ แทรกคำสั่ง JSP ไว้ในชุดคำสั่ง HTML ที่ใช้สร้างเว็บ และบันทึกไว้ในไฟล์นามสกุล JSP จากนั้นเก็บไว้ที่เซิร์ฟเวอร์ ที่สนับสนุนการทำงานของ JSP

3.2.3 JSP Servlet คือ Servlet ที่เกิดขึ้นจากชุดคำสั่ง JSP ที่ถูกคอมไพล์ไปเป็น JSP Servlet โดยขบวนการนี้เกิดโดยอัตโนมัติภายในเว็บเซิร์ฟเวอร์ที่สนับสนุน การทำงานของ JSP และ JSP Servlet นี้เองที่เป็นส่วนที่เกิดการประมวลผลบนเซิร์ฟเวอร์ ดังนั้นไฟล์นามสกุล JSP ที่ถูกเรียกใช้งาน จะต้องถูกคอมไพล์ไปเป็น JSP Servlet ก่อนในครั้งแรกของการเรียกใช้งาน หรือเมื่อมีการเปลี่ยนแปลงชุดคำสั่ง

3.2.4 การทำงานของ JSP



รูปที่ 3-1 รูปอธิบายการทำงานของ JSP

เมื่อเว็บเบราว์เซอร์ มีการร้องขอโดยผู้ใช้เพื่อเรียกข้อมูล JSP ที่อยู่บนเซิร์ฟเวอร์ ซึ่งก็หมายถึงไฟล์ที่มีนามสกุล JSP เว็บเซิร์ฟเวอร์จะทำการตรวจสอบว่า ไฟล์นั้นมีการคอมไพล์ไปเป็น JSP Servlet แล้วหรือยัง ถ้ายังไฟล์ JSP จะถูกคอมไพล์ โดยตัวเว็บเซิร์ฟเวอร์ ไปเป็น JSP Servlet แต่ถ้าไฟล์ถูกคอมไพล์แล้ว (ซึ่งเกิดขึ้นจากการใช้งานในครั้งแรก) เว็บเซิร์ฟเวอร์จะนำ JSP Servlet มาใช้งาน โดยการเรียกเพื่อประมวลผล ได้ผลลัพธ์อะไรก็จะเกิดเป็นข้อมูล ซึ่งเว็บเซิร์ฟเวอร์ จะส่งกลับไปให้ผู้เรียกใช้งาน และการประมวลผลของ JSP Servlet นี้เอง ที่ก่อให้เกิด ข้อมูลเว็บในแบบไดนามิก

3.3 จาวาบีน (JavaBean)

ในการเขียน JSP สำหรับระบบใหญ่ ๆ นักพัฒนาจะไม่นิยมใส่ java code ลงไปในไฟล์ JSP มากนัก สิ่งที่อยู่ในไฟล์มักจะเป็นเพียง HTML code และค่าของ variables ต่าง ๆ ที่ได้จาก JavaBeans เท่านั้น. ข้อดีของการใช้ JavaBean คือการง่ายต่อการเปลี่ยนแปลงลักษณะของ look-and-feel ของไฟล์ JSP ที่เป็นเช่นนั้น เพราะ ส่วนที่เป็น data (variables) กับส่วนที่เป็น presentation (HTML) จะอยู่ด้วยกันอย่างหลวม ๆ ดังนั้นเราจึงสามารถเปลี่ยนตัว presentation เมื่อไหร่ก็ได้ โดยตัว JavaBean ที่ใช้เก็บค่าของ variables ต่าง ๆ ยังคงเหมือนเดิม

สำหรับคนที่ไม่คุ้นกับ JavaBean ให้นึกถึง JavaBean เป็นลักษณะของกล่อง โดยกล่องนี้มีส่วนที่ใช้ติดต่อกับโลกภายนอกอยู่สองส่วนหลัก ๆ คือ

- ส่วนที่ใช้สำหรับรับข้อมูลต่าง ๆ โดยข้อมูลเหล่านี้จะถูกใช้เป็นตัวแปรในการควบคุมหรือเปลี่ยนแปลงคุณสมบัติและการทำงานของกล่อง (Setter)
- ส่วนที่ใช้สำหรับอ่านคุณสมบัติของกล่อง (Getter) ซึ่งโดยส่วนมาก(แต่ไม่ทั้งหมด) ก็คือค่าของตัวแปรต่าง ๆ ที่บรรจุอยู่ในกล่อง

ในการใช้งานกล่อง (JavaBean) ดังกล่าวร่วมกับ JSP เราสามารถที่จะสร้างกล่องขึ้นมาโดยใช้ `<jsp:useBean .../>`, ส่งค่าเข้าไปในกล่องโดยใช้ `<jsp:setProperty .../>` และอ่านค่าต่าง ๆ ที่อยู่ในกล่อง โดยใช้ `<jsp:getProperty .../>` tag แต่ก่อนที่เราจะเรียนรู้ถึงการใช้ tag เหล่านี้ เรามาพูดถึง JavaBean ในรายละเอียดอีกสักเล็กน้อยเสียก่อน

3.3.1 Bean Conventions สำหรับคนที่เคยเขียน application ที่เกี่ยวกับ JavaServlet หรือ RMI จะพบว่าแต่ละคลาสที่เราเขียนขึ้น อาจจะต้องมีการ extend คลาสใดคลาสหนึ่งก่อน เช่น HttpServlet (ในกรณีของการเขียน JavaServlet) หรืออาจต้องทำการ implement interface ต่าง ๆ เช่น Serializable (ในกรณีของการเขียน RMI) แต่สำหรับการเขียนคลาสให้กลายเป็น JavaBean แล้ว เราไม่จำเป็นต้องทำสิ่งต่าง ๆ ที่กล่าวมาข้างต้นเลย. JavaBean จริง ๆ แล้วก็คือ จาวาคลาสธรรมดา ๆ แต่ถูกเขียนขึ้นโดยใช้ naming และ design conventions ที่อยู่ใน JavaBean Specification เท่านั้น. Conventions หลัก ๆ ที่เราจะพูดถึงสำหรับการเขียน JavaBean เพื่อใช้ใน JSP มีดังต่อไปนี้คือ

3.3.2 The Bean Constructor ในการเขียน Bean ให้สามารถใช้ได้กับ `<jsp:useBean .../>` สิ่งหนึ่งที่ Bean ของเราจะต้องมีคือ ตัว constructor ที่ไม่ต้องการ argument สำหรับการ initialize ของ Bean * เราสามารถนำคลาสเหล่านี้ไปใช้กับ `<jsp:useBean ... />` ได้อย่างไรก็ตามคลาสนี้ยังไม่มีประโยชน์อะไรสำหรับไฟล์ JSP จริง ๆ ประโยชน์ที่เราจะได้จาก Bean ก็คือการส่งค่าต่าง ๆ เพื่อทำการควบคุมและเปลี่ยนแปลงคุณสมบัติของ Bean รวมไปถึงการอ่านค่าคุณสมบัติต่าง ๆ ของ Bean ซึ่งสิ่งเหล่านี้จะเกิดขึ้นได้โดยอาศัยสิ่งหนึ่งที่เรียกว่า Bean's Properties

3.2.3 Bean's Properties Bean's Properties คือตัวที่บอกว่าเราสามารถทำอะไรกับ JavaBean นั้นได้บ้าง [โดยส่วนมาก(แต่ไม่ทั้งหมด) ก็คือการเปลี่ยนและอ่านค่าต่าง ๆ ของ instance variables ที่บรรจุอยู่ข้างใน Bean นั้น ๆ] ซึ่งผลที่ได้ก็คือ การควบคุมและเปลี่ยนแปลงคุณสมบัติต่าง ๆ ของ Bean อย่างใดก็ตามบางที properties ที่เรากำหนดขึ้น อาจจะไม่ได้อะไรที่เกี่ยวข้องกับ instance variables ที่อยู่ข้างใน Bean นั้นเลย

JSP Container จะสามารถทราบได้ว่า JavaBean ของเรามี properties อะไรบ้าง ได้จากวิธีการที่เรียกว่า introspection หลักการก็คือ JSP Container จะทำการวิเคราะห์ฟังก์ชันที่เป็น public ของ Bean นั้น ซึ่งจะต้องเขียนขึ้นอย่างถูกต้องตามหลักของ JavaBean API โดยจะถือว่า property ใด ๆ ก็ตามจะเกิดขึ้นได้ก็ต่อเมื่อ Bean คลาสนั้นมี

- public ฟังก์ชันที่ใช้สำหรับควบคุมหรือเปลี่ยนแปลงค่าของ property นั้น (Setter) หรือ
- public ฟังก์ชันที่ใช้สำหรับอ่านค่าของ property นั้น (Getter) หรือ
- มีทั้งสองฟังก์ชันข้างต้น

ในการกำหนด property ขึ้นมาอันหนึ่ง เราจะต้องสร้าง public ฟังก์ชันที่ขึ้นต้นด้วยคำว่า set (Setter) ซึ่งใช้สำหรับควบคุมหรือเปลี่ยนแปลงค่าของ property นั้น และหรือ public ฟังก์ชันที่ขึ้นต้นด้วยคำว่า get (Getter) ซึ่งใช้สำหรับอ่านค่าของ property นั้น โดยรูปแบบโดยทั่วไปก็คือ

```
public void setPropertyName(PropertyType value);
```

```
public PropertyType getPropertyName();
```

ในการเขียน JavaBean เพื่อกำหนด properties ต่าง ๆ เราสามารถกำหนด properties ต่าง ๆ ขึ้นมา โดยไม่เกี่ยวข้องกับ instance variable ใด ๆ เลยก็ได้

3.2.4 Property Name Conventions โดยทั่วไป การเขียนชื่อของ properties ต่าง ๆ จะเป็นในลักษณะของลูกผสมคือ มีทั้งตัวใหญ่และตัวเล็ก ซึ่งจะขึ้นต้นด้วยตัวเล็กในคำแรกและจะตามด้วยตัวใหญ่ในคำถัด ๆ มา ยกตัวอย่างเช่น firstName, lastName เป็นต้น สำหรับตัว setter และ getter ของ properties ดังกล่าวก็จะเป็น setFirstName, setLastName ตามลำดับ

เราจะสังเกตเห็นว่า ชื่อของ properties และตัว setter, getter ของ properties เหล่านี้จะต่างกันที่ตัวแรก หลักจากคำว่า set หรือ get เท่านั้น ในการตีความ setter, getter เพื่อหา properties ต่าง ๆ ของ JavaBean โดย JSP Container, JSP Container จะทำการแปลง case ต่าง ๆ ให้เข้ากับชื่อของ properties ที่อยู่ใน `<jsp:getProperties .../>`, `<jsp:setProperty .../>` โดยอัตโนมัติ

3.2.5 Loading a Bean - `<jsp:useBean>` ก่อนที่เราจะสามารถใช้ JavaBean ใน JSP ได้ สิ่งแรกที่เราต้องทำคือ การโหลด instance ของ Bean เราสามารถโหลด instance ของ Bean เข้ามาในระบบได้โดยใช้ `<jsp:useBean .../>` โดยเราจะต้อง

- กำหนดชื่อที่จะใช้ในการอ้างถึง instance ของ Bean นี้ (id)
- กำหนดคลาสของ Bean ที่จะโหลด (class)
- ทำการเซต scope (หรือช่วงชีวิต) ของ Bean ว่าจะยาวนานแค่ไหน (scope)

ดังรูปแบบข้างล่าง

```
<jsp:useBean id="myBeanInstance" class="com.myPackage.BeanClass"
scope="page|request|session|application">
```

...body...

</jsp:useBean>

3.2.6 Initializing a Bean - <jsp:setProperty> ใน Bean Conventions เรากล่าวถึง Bean Constructor ที่ไม่ต้องการ arguments ซึ่งถูกใช้สำหรับ <jsp:useBean> อย่างไรก็ตามบางครั้ง Bean ที่เราใช้อาจต้องการค่าเริ่มต้นในการ initialize ซึ่งเราสามารถทำการเซตค่าต่าง ๆ ให้ Bean ได้โดยใช้ <jsp:setProperty .../>

ค่าที่เราจะเซตให้ Bean บางครั้งอาจมาจากค่าที่เรากำหนดขึ้น หรืออาจจะเป็นค่าที่ได้จาก request ของ client ซึ่งมาจาก element ต่าง ๆ ใน <FORM> tag ของ html ดังนั้นการเซตค่า Bean จึงถูกแบ่งออกเป็นสองแบบคือ

3.2.6.1 การเซตค่าให้ Bean โดยใช้ค่าที่เรากำหนดขึ้น เช่น

```
<jsp:useBean id="myBeanInstance" class="com.myPackage.BeanClass" scope="request">
<jsp:setProperty name="myBeanInstance" property="myProperty"
value="definedValue"/>
</jsp:useBean>
```

name คือ ชื่อของ Bean instance ที่เราต้องการอ้างอิงถึง

property คือ ชื่อของ property ที่เราต้องการเซตค่า

value คือ ค่าที่เราจะใส่เข้าไปให้ property นั้น

3.2.6.2 การเซตค่าให้ Bean โดยใช้ค่าที่ได้จาก element ต่าง ๆ ของ <FORM> tag เช่น

```
<jsp:useBean id="myBeanInstance" class="com.myPackage.BeanClass" scope="request">
<jsp:setProperty name="myBeanInstance" property="myProperty"
param="myFormElementName"/>
</jsp:useBean>
```

param คือ ชื่อของ element ที่อยู่ใน <FORM> tag ที่ส่ง request มาที่ไฟล์ JSP ที่มี

<jsp:useBean> นี้

หลังจากที่เราทำการโหลด และเซตค่าของ property ต่าง ๆ ใน Bean แล้ว เราสามารถดูผลที่เกิดขึ้นได้โดยใช้ <jsp:getProperty name="myBeanInstance" property="myProperty"/>

ค่าที่ได้จาก <jsp:getProperty> อาจจะเป็นค่าของ instance variables ของ Bean ที่เปลี่ยนไปหลังจากที่เราใช้ <jsp:setProperty> หรือบางครั้ง อาจจะเป็นลิสต์ของตารางรายชื่อ ในกรณีที่ว่า property ที่เราเซตผ่าน <jsp:setProperty> เป็นตัว trigger ให้ Bean ส่ง query ไปที่ดาตาเบส เป็นต้น